



Yenepoya Institute of Arts, Science, Commerce and Management

High Level Design

on

Predictive Analysis of Healthcare Success

Dataset: **Heart Disease Prediction**

Student Name: Misab

Industry Mentor: Sumith Kumar Shukla

Guided by: Raksha

Table of Contents

1. Introduction	3
1.1. Scope of the Document	3
1.2. Intended Audience	3
1.3. System Overview	3
2. Architecture Overview	4
2.1. Architectural Style	4
2.2. High-Level Architecture Diagram Description	4
2.3. Key Architectural Decisions	4
3. System Components	5
3.1. Data Ingestion Layer	5
3.2. Preprocessing & Feature Engineering Layer	5
3.3. Model Training Layer	5
3.4. Evaluation & Reporting Layer	5
3.5. Visualization Layer	5
4. Technology Stack	6
5. Data Flow	6
6. Security & Compliance	7
7. Non-Functional Requirements	7
7.1. Performance	7
7.2. Scalability	7
8. Deployment View	8
9. 7.3. Reliability	7
Risks & Mitigations	8
10. References	8

1. Introduction

1.1 Scope of the Document

- **Architecture Overview:** The end-to-end structural design of the ML pipeline from raw data ingestion to prediction output.
- **Component Responsibilities:** High-level definition of each processing layer and its role within the system.
- **Technology Stack:** Libraries, frameworks, and runtime environment selected for the project.
- **Data Flow:** The journey of data through ingestion, preprocessing, training, evaluation, and visualization stages.
- **Security & Compliance:** Data privacy considerations and access control strategy.
- **Non-Functional Requirements:** Performance, scalability, and reliability expectations at an architectural level.
- **Deployment View:** How the system is packaged and executed in a local development environment.

1.2 Intended Audience

- **System Architects & Designers:** For understanding the overall structural blueprint before implementation.
- **Data Scientists / ML Engineers:** To align implementation choices with the approved architecture.
- **Quality Assurance (QA) Teams:** For creating high-level test plans that validate architectural correctness.
- **Academic Reviewers & Mentors:** To evaluate design soundness, technology appropriateness, and alignment with project objectives.

Project Stakeholders: For a non-technical overview of how the system is structured and what it delivers.

1.3 System Overview

The system is a supervised machine learning pipeline that ingests a structured clinical dataset (heart.csv — 1,025 patient records, 14 features), applies preprocessing, trains multiple classification models, evaluates their predictive accuracy, and renders visual insights. The best-performing model — Random Forest Classifier — achieves 98.54% accuracy in predicting the presence or absence of heart disease. The entire pipeline runs within a Jupyter Notebook environment without any external server, database, or API dependency.

2. Architecture Overview

2.1 Architectural Style

The system follows a Linear Pipeline (Pipe-and-Filter) architectural pattern. Each stage in the pipeline receives data from the previous stage, applies a well-defined transformation or computation, and passes results to the next stage. There are no feedback loops between stages in the primary workflow, ensuring unidirectional data flow and clear separation of concerns.

- Pattern: Linear Pipeline / Pipe-and-Filter
- Execution Model: Sequential, synchronous, single-threaded Jupyter kernel execution
- Persistence Model: Stateless per-session — all objects reside in kernel memory; no disk writes

Interface Model: Single-user notebook interface; no client-server separation required

2.2 High-Level Architecture Diagram Description

The architecture consists of five vertically stacked layers, each building on the output of the previous:

Layer	Name	Input	Output
L1	Data Ingestion	heart.csv (disk)	Raw DataFrame (1025 × 14)
L2	Preprocessing & Feature Engineering	Raw DataFrame	X_train, X_test, y_train, y_test arrays
L3	Model Training	Training arrays	Fitted model objects
L4	Evaluation & Reporting	Test arrays + model objects	Accuracy scores, confusion matrix
L5	Visualization	Metrics + model objects + DataFrame	Inline matplotlib figures

2.3 Key Architectural Decisions

- Notebook-First Design: Jupyter Notebook chosen as the execution environment to enable iterative exploration, inline visualization, and reproducible analysis — ideal for academic and research contexts.
- No External Database: The dataset is small enough (< 1 MB) to reside entirely in memory; a database would add unnecessary infrastructure overhead.
- Multiple Model Comparison: Three algorithms (Decision Tree, Random Forest, Logistic Regression) are trained and compared in a single pipeline to identify the best performer objectively.

- Scikit-learn Unified API: All models implement the same `.fit()` / `.predict()` interface, enabling the comparison loop to be written generically without model-specific branching.

Fixed Random Seed: `random_state=42` applied globally to ensure reproducibility of splits and model outputs across all sessions

3. System Components

3.1 Data Ingestion Layer

- Responsibility: Load the raw heart disease dataset from local filesystem into a pandas DataFrame.
- Primary Operation: `pd.read_csv('heart.csv')` — reads 1,025 rows $\times$ 14 columns.
- Validation Checks: `df.shape` (confirm dimensions), `df.dtypes` (confirm all numeric), `df.isnull().sum()` (confirm zero nulls).
- Output Contract: A clean, fully populated DataFrame with no missing values, passed to the Preprocessing Layer.

3.2 Preprocessing & Feature Engineering Layer

- Responsibility: Separate features from target, and partition the dataset into training and test subsets.
- Feature Separation: `X = df.drop('target', axis=1)` $\rightarrow$ 13 clinical input features; `y = df['target']` $\rightarrow$ binary labels.
- Train-Test Split: `train_test_split(X, y, test_size=0.2, random_state=42)` $\rightarrow$ 820 training samples / 205 test samples.
- No Scaling Required: All three models used (Decision Tree, Random Forest, Logistic Regression) are either scale-invariant or perform adequately without feature normalization on this dataset.
- Output Contract: Four arrays — `X_train`, `X_test`, `y_train`, `y_test` — passed to the Model Training Layer.

3.3 Model Training Layer

- Responsibility: Instantiate, configure, and fit classification models on the training data.
 - `DecisionTreeClassifier(max_depth=5, random_state=42)` — shallow tree to prevent overfitting.
 - `RandomForestClassifier(n_estimators=100)` — ensemble of 100 trees for highest accuracy.
 - `LogisticRegression()` — linear baseline model for comparison.
- Training Call: `model.fit(X_train, y_train)` for each classifier.
- Output Contract: Fitted model objects retained in kernel memory, passed to Evaluation and Visualization layers.

3.4 Evaluation & Reporting Layer

- Responsibility: Quantify each model's predictive performance on unseen test data.
- Prediction: `y_pred = model.predict(X_test)` for each trained model.
- Metrics Computed: `accuracy_score(y_test, y_pred)` and `confusion_matrix(y_test, y_pred)`.

- Multi-Model Comparison Loop: Iterates over all three classifiers, trains each with default splits, and prints a formatted accuracy summary table.
- Output Contract: Numeric accuracy values and confusion matrix arrays passed to Visualization Layer.

3.5 Visualization Layer

- Responsibility: Render all analytical charts inline within the Jupyter Notebook.
- Confusion Matrix Heatmap: `ConfusionMatrixDisplay(cm, display_labels=['No Disease','Disease']).plot(cmap='Blues')`.
- Feature Importance Chart: Horizontal bar chart of `model.feature_importances_` sorted by magnitude.
- Class Distribution Plot: Count plot of target column (526 disease vs. 499 no-disease).
- Sex Distribution Pie Chart: Pie chart of patient sex breakdown using `df['sex'].value_counts()`.
- Output Contract: Inline matplotlib figures rendered in the notebook output cells.

4. Technology Stack

Category	Technology	Version	Purpose
Runtime	Python	3.14	Core programming language for the entire pipeline
Execution Env.	Jupyter Notebook	Latest	Interactive development and inline visualization environment
Data Handling	pandas	3.0.2	DataFrame operations — load, inspect, split, filter
Numerical Ops.	NumPy	2.4.4	Underlying array operations for ML computations
ML Framework	scikit-learn	1.8.0	DecisionTree, RandomForest, LogisticRegression, metrics
Visualization	matplotlib	3.10.8	All chart rendering — confusion matrix, bar, pie charts
Date Utilities	python-dateutil	2.9.0	Date parsing support for pandas internals
OS	Windows (local machine)	—	Host operating system for Jupyter kernel execution

5. Data Flow

The data flows through the system in a strict top-to-bottom sequential pipeline with no branching or feedback loops in the primary execution path:

Step	From	To	Data Transferred	Mechanism
1	Filesystem (heart.csv)	Data Ingestion Layer	Raw CSV file (1,025 rows × 14 cols)	pd.read_csv()
2	Data Ingestion Layer	Preprocessing Layer	Validated pandas DataFrame	Python variable assignment
3	Preprocessing Layer	Model Training Layer	X_train (820×13), y_train (820,)	train_test_split()
4	Preprocessing Layer	Evaluation Layer	X_test (205×13), y_test (205,)	train_test_split()
5	Model Training Layer	Evaluation Layer	Fitted model objects	Python variable (model)
6	Evaluation Layer	Visualization Layer	y_pred array, confusion matrix, accuracy	Python variables
7	Model Training Layer	Visualization Layer	model.feature_importances_ array	Python attribute access
8	Data Ingestion Layer	Visualization Layer	df['sex'], df['target'] Series	DataFrame column access
9	Visualization Layer	Notebook Output	Rendered matplotlib figures (inline)	plt.show()

Key Data Flow Characteristics:

- Unidirectional: Data moves strictly forward through the pipeline; no stage reads from a downstream stage.
- In-Memory Only: No intermediate results are serialized to disk; all transfers are Python variable assignments.
- Shared Reference: The original DataFrame (df) is referenced by both the Preprocessing Layer and the Visualization Layer without copying, conserving memory.
- Immutable Source: heart.csv is never modified; the pipeline is purely read-only with respect to source data.

6. Security & Compliance

6.1 Data Privacy

- The heart.csv dataset is sourced from the UCI Machine Learning Repository and contains fully anonymized patient records — no names, national IDs, contact information, or any directly identifiable attributes are present.
- No data is transmitted over any network; all processing occurs locally on the developer's machine.

- No user-generated data, login credentials, or sensitive information is collected or stored at any point in the pipeline.

6.2 Access Control

- The system runs as a local Jupyter Notebook accessible only to the authenticated OS user on the host machine.
- No web-facing endpoints, APIs, or shared services are exposed — the attack surface is effectively zero.
- File system access is read-only for the source dataset; the pipeline does not write any output files.

6.3 Compliance Considerations

- Academic Use: The dataset is publicly licensed for research and educational purposes under UCI repository terms.
- GDPR / HIPAA Inapplicability: Since no real patient data is used and no data is transmitted or stored persistently, formal healthcare data regulations do not apply to this academic implementation.
- Reproducibility Compliance: Fixed random seeds and version-pinned dependencies ensure that results are fully reproducible and auditable.

7. Non-Functional Requirements

7.1 Performance

- Data Load Time: heart.csv (< 1 MB, 1,025 rows) loads into memory in under 100 milliseconds on any modern machine.
- Training Time: Decision Tree and Logistic Regression complete training in under 1 second on 820 samples. Random Forest (100 estimators) completes in 2–4 seconds.
- Prediction Time: All model predictions on 205 test samples complete in under 50 milliseconds — suitable for near-real-time inference.
- Visualization Rendering: All matplotlib charts render inline within 1–2 seconds per figure.
- End-to-End Pipeline: Full execution of all notebook cells (load → train → evaluate → visualize) completes in under 30 seconds.

7.2 Scalability

- Current Capacity: Optimized for datasets up to approximately 100,000 rows with the existing in-memory design.
- Horizontal Scaling Path: For larger datasets, the pipeline can be migrated to Apache Spark with MLlib, or scikit-learn's joblib parallel backend can be enabled for RandomForest training.
- Feature Scaling Path: Additional clinical features can be added to heart.csv without any architectural changes — only the preprocessing validation logic would need updating.
- Model Scaling Path: New algorithms (e.g., XGBoost, SVM, Neural Networks) can be added to the comparison loop without modifying any other layer.

7.3 Reliability

- **Reproducibility:** `random_state=42` guarantees identical results on every execution, ensuring that any reviewer can independently verify the reported 98.54% Random Forest accuracy.
- **Zero Null Tolerance:** The validation step (`df.isnull().sum()`) explicitly confirms data completeness before any processing begins.
- **Fault Isolation:** Each notebook cell is independently executable — a failure in the Visualization Layer does not affect the Model Training or Evaluation layers.
- **Dependency Stability:** All library versions are pinned (scikit-learn 1.8, pandas 3.0, matplotlib 3.10) to prevent breaking changes from upstream updates.

8. Deployment View

The system is deployed as a self-contained local development environment with no external dependencies beyond the host machine:

Component	Deployment Unit	Host	Notes
Source Data	heart.csv	Local filesystem	Static read-only file; placed in same directory as notebook
Execution Engine	Jupyter Notebook kernel	Local machine (Windows)	Python 3.14 kernel; all computation in-process
ML Pipeline Code	Untitled__1_.ipynb	Local filesystem	All pipeline stages encoded as sequential notebook cells
Dependencies	pip packages	Python environment	pandas, scikit-learn, matplotlib, numpy — installed via pip
Output	Notebook cell outputs	In-memory / .ipynb file	Charts and metrics rendered inline; saved on notebook save

8.1 Deployment Steps

- **Step 1 — Install Dependencies:** `pip install pandas scikit-learn matplotlib`
- **Step 2 — Place Dataset:** Copy heart.csv into the same directory as the .ipynb notebook file.
- **Step 3 — Launch Jupyter:** Run jupyter notebook from the project directory to open the notebook server.
- **Step 4 — Execute Pipeline:** Run all cells sequentially (Kernel → Restart & Run All) to reproduce all results.

- Step 5 — View Outputs: All metrics and visualizations are rendered inline within the notebook interface.

8.2 Environment Requirements

- Operating System: Windows, macOS, or Linux (any OS supporting Python 3.8+)
- Python Version: 3.8 or higher (tested on Python 3.14)
- RAM: Minimum 512 MB free memory (dataset is < 1 MB; models are lightweight)
- Storage: Minimum 50 MB for Python environment and dependencies

9. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation Strategy
Dataset too small to generalize to real clinical populations	Medium	High	Document scope as academic; recommend validation on larger datasets (e.g., MIMIC-III) before clinical use
Model overfitting (Decision Tree)	Medium	Medium	max_depth=5 cap applied; Random Forest ensemble used as the primary model to reduce variance
Library version incompatibility breaks pipeline	Low	Medium	All versions pinned in requirements; tested on Python 3.14 with scikit-learn 1.8
Kernel state loss on crash (data not saved)	Low	Low	Pipeline re-executes fully in < 30s; no persistent state required
Class imbalance affecting model bias	Low	Medium	Dataset is nearly balanced (526 vs 499); no SMOTE needed; verified via <code>df['target'].value_counts()</code>
Incorrect feature interpretation without domain knowledge	Medium	Medium	All 14 features documented with clinical descriptions in LLD data model table

10. References

- scikit-learn Documentation: <https://scikit-learn.org/stable/>
- pandas Documentation: <https://pandas.pydata.org/docs/>
- matplotlib Documentation: <https://matplotlib.org/stable/>

- Heart Disease Dataset: UCI Machine Learning Repository — Cleveland Heart Disease Dataset
- Python 3.14 Reference: <https://docs.python.org/3/>
- Jupyter Notebook Documentation: <https://jupyter-notebook.readthedocs.io/>
- Random Forest Algorithm — Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.